

Password-Derived Key Image Encryption Using Frequency-Domain Convolution: Analysis and Performance Evaluation

M.C. Zuriel López Sosa, Dra. Bárbara Emma Sánchez Rinza, and Dr. Carlos Ignacio Robledo Sánchez

Benemerita Universidad Autonoma de Puebla
zuriel.lopezs@correo.buap.mx, brinza@fcfm.buap.mx, crobledo@fcfm.buap.mx

Abstract. Image encryption is essential for secure digital communication and the protection of visual data. This paper presents an encryption method based on the Fourier Transform (FT), in which both the original image and a key, cryptographically generated from a password, are transformed into the frequency domain. The key is derived using PBKDF2-HMAC-SHA256 and expanded with SHAKE-256 to exactly the size of the image, removing the need for an external key file. The encryption process applies the Fast Fourier Transform (FFT) to both representations and multiplies them element-wise; the encrypted image is reconstructed using the Inverse FFT (IFFT) and stored as a 16-bit-per-channel PNG with the normalization metadata embedded in the file. To evaluate the security and effectiveness of the method, statistical analyses (histograms, entropy, and PSNR) and robustness tests against Gaussian noise, resizing, and incorrect-password attacks were performed. The results show that the encrypted image exhibits high randomness and low correlation with the original, that decryption with the correct password recovers the original image exactly (infinite PSNR), and that the method is highly sensitive to alterations of the key, the size of the encrypted image, and the password used.

Keywords: Image encryption, Fourier Transform, digital security, frequency-domain encryption, key derivation, PBKDF2, SHAKE-256, cryptographic analysis.

1 Introduction

In the digital age, information security is a priority, particularly regarding the protection of images, which often contain sensitive data. Digital images, due to their visual nature, present an additional challenge for traditional spatial-domain encryption methods. Frequency-domain encryption techniques, such as those based on the Fourier Transform (FT), have gained attention for their ability to effectively conceal data patterns, making decryption without the proper key difficult [1, 2]. Existing alternatives include random phase encoding schemes in the optical domain [3] and chaos-based schemes [4, 5], which pursue similar randomization goals through nonlinear spatial-domain dynamics.

Image encryption based on FT, and specifically on the Fast Fourier Transform (FFT), offers an efficient solution to the problem of protecting images over unsecured channels. This work presents a method that transforms both the original image and a key into the frequency domain and combines them through element-wise multiplication. Unlike schemes that require distributing a separate key image alongside the encrypted data, the key used here is derived from a **password** using standard cryptographic functions, so that only the encrypted file and the password are needed to recover the original image, in line with the formal cryptographic security notions requiring that an attacker without the correct key be unable to distinguish the ciphertext from random noise [16].

The aim of this paper is to present and evaluate this method, examining its effectiveness and security. Performance evaluation is conducted through statistical tests (histogram comparison, entropy, and PSNR) and robustness tests against potential attacks.

The structure of this article is as follows: Section 2 describes the proposed algorithm, detailing the encryption and decryption process. Section 3 presents the tests performed and the analysis of results. Finally, Section 4 concludes with a discussion of the findings.

2 Methodology

The algorithm is based on the **Fourier Transform convolution property**, which states that the Fourier transform of the convolution of two functions equals the product of their respective Fourier transforms:

$$\mathcal{F}(f * g) = \mathcal{F}(f) \cdot \mathcal{F}(g)$$

Isolating the convolution term and applying the Inverse Fourier Transform ($\mathcal{F}^{-1}$) on both sides gives:

$$f * g = \mathcal{F}^{-1}(\mathcal{F}(f) \cdot \mathcal{F}(g))$$

This implies that convolution in the spatial domain can be computed indirectly through multiplication in the frequency domain, followed by the Inverse Transform. In practice, both transforms are computed using the Fast Fourier Transform (FFT) algorithm [6], which reduces the computational complexity from $O(N^2)$ to $O(N \log N)$.

Throughout this article, f denotes the **original image** to be encrypted and g denotes the **key image**. Encryption is defined as the convolution $f * g$: this convolution, computed in the frequency domain as described above, **is** the encrypted image. To recover the original image, it suffices to solve for f in the convolution property, dividing in the frequency domain by $\mathcal{F}(g)$ instead of multiplying:

$$f = \mathcal{F}^{-1} \left(\frac{\mathcal{F}(f * g)}{\mathcal{F}(g)} \right)$$

This is precisely the relationship instantiated by the encryption process (Section 2.2) and the decryption process (Section 2.3), respectively: the former produces $f * g$ from f and g ; the latter recovers f from $f * g$ and g . Since the images

used are in RGB format, all operations are performed independently on each of the three color channels (Red, Green, and Blue), i.e., f , g , and $f * g$ are handled channel by channel.

2.1 Deriving the key g from a password

Instead of requiring g to come from an external key image, g is cryptographically generated from a password. This design follows Kerckhoffs's principle [7], which states that the security of a cryptographic system should rely solely on the secrecy of the key (here, the password) and not on the secrecy of the algorithm or the possession of an additional file. The derivation follows the same principle used by modern stream ciphers and by password-based key derivation functions [9, 10]: deriving a short key and expanding it into a pseudorandom stream of the required length.

1. A random 16-byte salt is generated at encryption time.
2. The password and salt are processed with **PBKDF2-HMAC-SHA256** [8] using 310,000 iterations, producing 32 derived bytes.
3. Those 32 bytes are expanded with **SHAKE-256** (an extendable-output function from the SHA-3 family [11]) to obtain exactly $\text{height} \times \text{width} \times 3$ bytes: these bytes constitute g , generated at exactly f 's dimensions, without requiring any resizing.

2.2 Encryption process: obtaining $f * g$

1. **Inputs:** the original image f to encrypt and a password.
2. **Key derivation:** g is obtained as described above.
3. **Fourier Transform:** the FFT is applied to each channel of f and of g , giving $\mathcal{F}(f)$ and $\mathcal{F}(g)$.
4. **Element-wise multiplication:** $\mathcal{F}(f) \cdot \mathcal{F}(g)$ is computed.
5. **Inverse Fourier Transform:** the IFFT is applied to the previous result, yielding $f * g$ in the spatial domain.
6. **Normalization and storage:** per-channel Min-Max normalization is applied to $f * g$ to scale its values to a 16-bit range $([0, 65535])$, since the convolution result has a much larger value range than a conventional image. The minimum and maximum used, together with the salt and iteration count, are embedded as metadata inside the encrypted PNG file itself (as a JSON-encoded `tEXt` chunk), so that the resulting file — which stores $f * g$ quantized to 16 bits — is self-contained. This file is the **encrypted image**.

2.3 Decryption process: recovering f from $f * g$

1. **Inputs:** the encrypted image (storing $f * g$ quantized to 16 bits) and the password.
2. **Reading metadata:** the salt, iteration count, and per-channel min/max ranges are read from the file itself.

3. **Key re-derivation:** the process from Section 2.1 is repeated with the same salt, regenerating g identically to the one used during encryption.
4. **Reversing normalization:** the 16-bit values are converted back to the real scale of $f * g$ using the stored ranges.
5. **Fourier Transform:** the FFT is applied to $f * g$ (now denormalized) and to g , giving $\mathcal{F}(f * g)$ and $\mathcal{F}(g)$.
6. **Element-wise division:** f is recovered by dividing both frequency-domain representations:

$$f \approx \mathcal{F}^{-1} \left(\frac{\mathcal{F}(f * g)}{\mathcal{F}(g) + \epsilon} \right)$$

with $\epsilon = 1 \times 10^{-10}$ to avoid division by zero. The result of this operation is the **decrypted image**.

7. **Rounding and clipping:** the result is rounded and clipped to the valid $[0, 255]$ range to obtain the final 8-bit image.

The resulting storage format corresponds to a 16-bit-per-channel image, capable of representing the full range of values required by the normalization described above.

3 Analysis and Performance Evaluation

This section presents the analysis and performance evaluation of the proposed method. The evaluation was performed on a 3024×4032 -pixel test photograph and consists of three stages: visual inspection, statistical analysis, and robustness testing against attacks.

3.1 Visual inspection

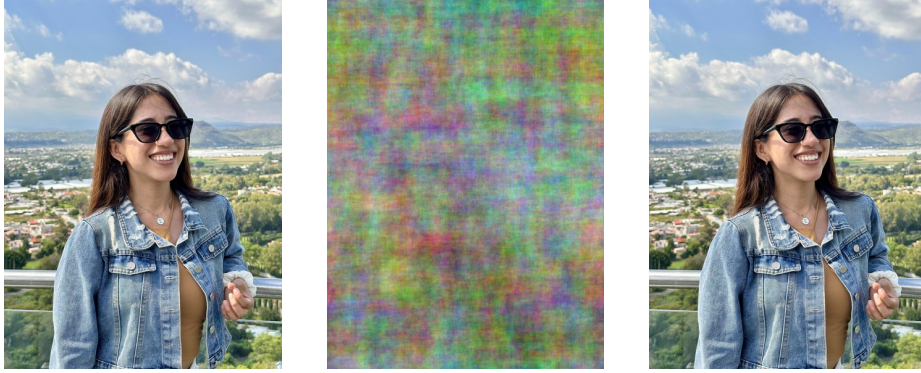


Fig. 1. Left to right: original image f , encrypted image $f * g$ (8-bit preview of the actual 16-bit data), and decrypted image (recovery of f) obtained by decrypting with the correct password.

The encrypted image $f * g$ shows no recognizable trace of f 's content, while the decrypted image is, as quantified in Section 3.3, identical to f .

3.2 Statistical analysis

Histogram analysis. Intensity histograms are a standard tool for analyzing the pixel-value distribution of an image [13]. Histograms of f and $f * g$ were computed for each color channel (Fig. 2). The original image's histogram shows pronounced peaks associated with regions of uniform color (sky, skin, clothing). The encrypted image's histogram, in contrast, takes on a smooth, unimodal, Gaussian-like shape, consistent with the central limit theorem applied to the convolution of two statistically independent signals: the peaked structure of the original disappears entirely, indicating that the encryption process effectively conceals the original intensity distribution.

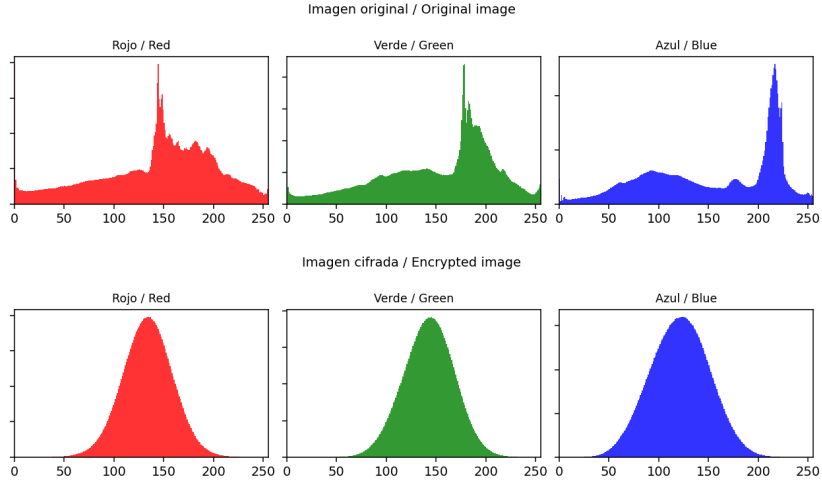


Fig. 2. Per-channel (R, G, B) histograms of the original image (top) and the encrypted image (bottom).

Entropy analysis. The average Shannon entropy [12] (across the three channels) of f is 7.6512, while that of $f*g$ is 6.7715. This slight decrease, also observed in other FT-based encryption schemes, is explained by the fact that Min-Max normalization concentrates values into a bell-shaped distribution rather than uniformly saturating all 256 possible levels; nonetheless, entropy remains high, confirming that the encryption produces an image with high apparent randomness.

PSNR evaluation. The Peak Signal-to-Noise Ratio (PSNR) [14] between f and the image recovered by decrypting $f * g$ with the correct password was computed. The result obtained was:

$$\text{MSE} = 0.0 \quad \text{PSNR} = \infty$$

That is, the recovered image is **bit-for-bit identical** to the original. This confirms that, by using 16 bits per channel with the normalization ranges stored as exact metadata, the encryption and decryption process introduces no information loss whatsoever.

3.3 Robustness against attacks

Beyond histogram, entropy, and PSNR testing, the image encryption literature frequently evaluates pixel-level sensitivity through randomness tests such as NPCR and UACI [15]. In this work, robustness was evaluated directly against three concrete attacks: Gaussian noise, resizing, and an incorrect password.

Gaussian noise attack. Gaussian noise was added directly to $f * g$ (the 16-bit encrypted data), at three standard deviation (σ) levels expressed as a fraction of the 16-bit range (0.05%, 0.25%, and 0.5%, equivalent to $\sigma \approx 32.8$, 163.8, and 327.7 in 16-bit units). The results are shown in Fig. 3 and Table 1.



Fig. 3. Decrypted image after adding low, medium, and high levels of Gaussian noise to the encrypted file, with their corresponding PSNR.

Table 1. PSNR of the decrypted image after the Gaussian noise attack.

Level	σ (16-bit)	MSE	PSNR (dB)
Low	32.8	1.67	45.90
Medium	163.8	33.92	32.83
High	327.7	188.43	25.38

As expected, PSNR decreases as the noise level increases. However, even at the highest level tested, the recovered image remains clearly recognizable (Fig. 3), indicating that the method reasonably tolerates small perturbations of the encrypted file without losing the usefulness of the recovered image.

Resizing attack. The effect of applying bilinear interpolation to change the scale of $f * g$ (16-bit) before decryption was evaluated (regenerating g at the new size in each case, since g is deterministically derived from the password and the image's dimensions).

Downscaling. $f * g$ was reduced to 1%, 5%, and 10% of its original size. In all three cases, decryption resulted in a **fully saturated** image (every pixel white, value 255; Fig. 4). This is explained by the fact that the zero-frequency term (the DC component) of the encrypted data retains a magnitude associated with the original image size, while the key is regenerated at a much smaller scale;

the mismatch between these magnitudes causes the frequency-domain division result to overflow and saturate when clipped to the $[0, 255]$ range.

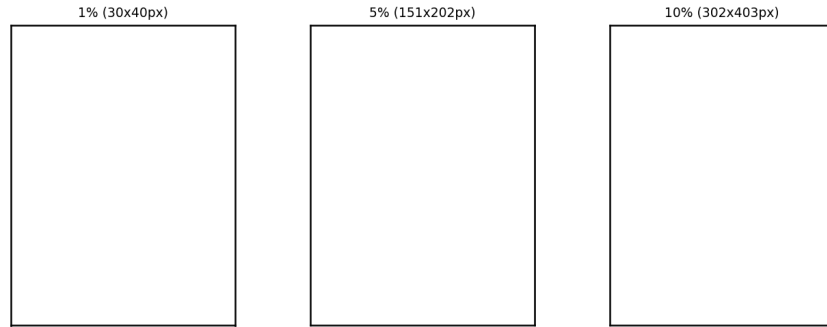


Fig. 4. Decryption result after reducing the encrypted image to 1%, 5%, and 10% of its original size: total saturation (white) in all three cases.

Upscaling. The encrypted image was enlarged by 1%, 5%, and 10%. In all three cases, the result was completely unrecognizable colored noise (Fig. 5), with no trace of the original content. Interpolation introduces artifacts that disrupt the frequency-domain representation, preventing correct decryption.

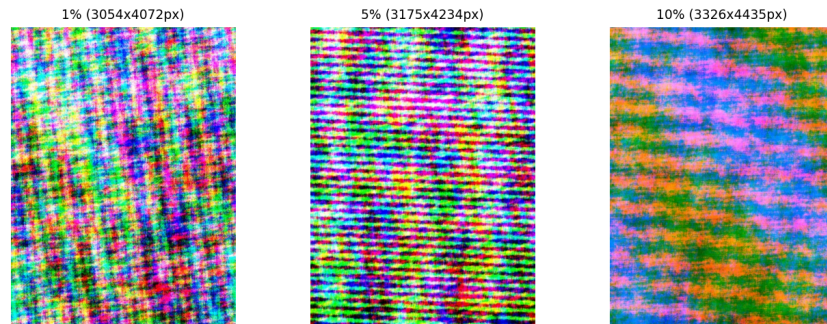


Fig. 5. Decryption result after enlarging the encrypted image by 1%, 5%, and 10%: unrecognizable noise in all three cases.

In both cases (downscaling and upscaling), the method proves to be highly sensitive to any geometric alteration of the encrypted file, a desirable property for detecting unauthorized tampering.

Decryption with an incorrect password. Decryption was attempted using a password different from the one used during encryption. The result, shown in Fig. 6, is completely unrecognizable colored noise:

$$\text{MSE} = 13727.85 \quad \text{PSNR} = 6.75 \text{ dB}$$

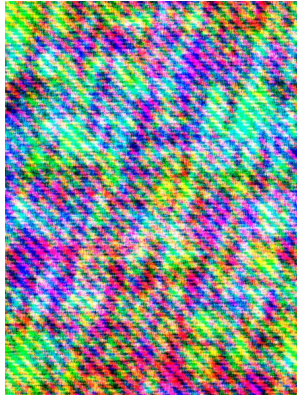


Fig. 6. Result of attempting to decrypt the image with an incorrect password.

This result confirms that the scheme critically depends on the exact password: even a different password produces a completely unusable reconstruction, reinforcing the security of the method.

4 Conclusions

The analysis demonstrates that the proposed frequency-domain convolution encryption method, with a password-derived key, effectively transforms an image's content into a randomized-looking representation that is difficult to distinguish, visually and statistically, from noise. Statistical evaluations confirm high entropy and a complete loss of the original histogram structure in the encrypted image, while decryption with the correct password recovers the original image **exactly** (infinite PSNR), unlike equivalent schemes based on 8-bit normalization, which introduce irreversible information loss.

Robustness tests show that the method tolerates moderate levels of Gaussian noise while keeping the recovered image recognizable, but is extremely sensitive to geometric transformations (resizing) and to any deviation in the password used, both desirable properties for detecting unauthorized tampering with the encrypted file. Overall, the scheme offers a strong level of security while also removing the need to distribute a separate key file, making it practical for secure image transmission.

References

1. Refregier, P., Javidi, B.: Optical image encryption based on input plane and Fourier plane random encoding. *Optics Letters* 20(7), 767–769 (1995).
2. Unnikrishnan, G., Joseph, J., Singh, K.: Optical encryption system that uses phase conjugation in a photorefractive crystal. *Applied Optics* 37(35), 8181–8186 (1998).
3. Javidi, B., Zhang, G., Li, J.: Experimental demonstration of the random phase encoding technique for image encryption and security verification. *Optical Engineering* 35(9), 2506–2512 (1996).
4. Chen, G., Mao, Y., Chui, C.K.: A symmetric image encryption scheme based on 3D chaotic cat maps. *Chaos, Solitons & Fractals* 21(3), 749–761 (2004).
5. Zhou, Y., Bao, L., Chen, C.L.P.: A new 1D chaotic system for image encryption. *Signal Processing* 97, 172–182 (2014).
6. Cooley, J.W., Tukey, J.W.: An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation* 19(90), 297–301 (1965).
7. Kerckhoffs, A.: La cryptographie militaire. *Journal des sciences militaires* 9, 5–38 (1883).
8. Moriarty, K., Kaliski, B., Rusch, A.: PKCS #5: Password-Based Cryptography Specification, Version 2.1. RFC 8018 (2017).
9. Turan, M.S., Barker, E., Burr, W., Chen, L.: Recommendation for Password-Based Key Derivation, Part 1: Storage Applications. NIST Special Publication 800-132 (2010).
10. Percival, C.: Stronger key derivation via sequential memory-hard functions. In: BSDCan 2009 (2009).
11. National Institute of Standards and Technology: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. FIPS PUB 202 (2015).
12. Shannon, C.E.: A mathematical theory of communication. *The Bell System Technical Journal* 27(3), 379–423 (1948).
13. Gonzalez, R.C., Woods, R.E.: *Digital Image Processing*, 4th ed. Pearson (2018).
14. Hore, A., Ziou, D.: Image quality metrics: PSNR vs. SSIM. In: 20th International Conference on Pattern Recognition (ICPR), pp. 2366–2369 (2010).
15. Wu, Y., Noonan, J.P., Agaian, S.: NPCR and UACI randomness tests for image encryption. *Cyber Journals: Journal of Selected Areas in Telecommunications*, 31–38 (2011).
16. Bellare, M., Desai, A., Pointcheval, D., Rogaway, P.: Relations among notions of security for public-key encryption schemes. In: CRYPTO 1998, LNCS 1462, pp. 26–45 (1998).